

CSS311 · Parallel and Distributed Computing

LAB 1

Name : Vineeth M N

Roll No : 2024bcs0178

Codes and output Screenshots

Q1

a.

```
C hello.c > ...
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Hello World\n");
6      return 0;
7  }
8  |
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• [vini@asus SEM5]$ cd "/home/vini/Documents/SEM5/" && gcc hello.c -o hello && "/home/vini/Documents/SEM5/"hello
Hello World
• [vini@asus SEM5]$ cd "/home/vini/Documents/SEM5/" && gcc hello.c -o hello && "/home/vini/Documents/SEM5/"hello
Hello World
○ [vini@asus SEM5]$
```

B.

```
C hello_parallel.c > ...
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(void)
5  {
6      #pragma omp parallel
7      {
8          printf("Hello World\n");
9      }
10
11      return 0;
12  }
13
```

Output

```
PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[vini@asus SEM5]$ cd "/home/vini/Documents/SEM5/" && gcc hello.c -o hello && "/home/vini/Documents/SEM5/"hello
• [vini@asus SEM5]$ cd "/home/vini/Documents/SEM5/" && gcc hello.c -o hello && "/home/vini/Documents/SEM5/"hello
Hello World
• [vini@asus SEM5]$ cd "/home/vini/Documents/SEM5/" && gcc hello_parallel.c -o hello_parallel && "/home/vini/Documents/SEM5/"hello_parallel
Hello World
○ [vini@asus SEM5]$
```

Q2.

a.

```
C hello_thread_id.c > ...
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(void)
5  {
6      #pragma omp parallel
7      {
8          int thread_id = omp_get_thread_num();
9
10         printf("Hello World from thread %d\n", thread_id);
11     }
12
13     return 0;
14 }
15
```

Output

```
[vini@asus SEM5]$ gcc -fopenmp hello_thread_id.c -o hello_thread_id
[vini@asus SEM5]$ ./hello_thread_id
Hello World from thread 3
Hello World from thread 14
Hello World from thread 13
Hello World from thread 15
Hello World from thread 6
Hello World from thread 8
Hello World from thread 9
Hello World from thread 10
Hello World from thread 0
Hello World from thread 11
Hello World from thread 2
Hello World from thread 7
Hello World from thread 1
Hello World from thread 4
Hello World from thread 12
Hello World from thread 5
[vini@asus SEM5]$
```

B.

```
C hello_timing.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  int main(int argc, char *argv[])
6  {
7      long long k = 10;
8
9      if (argc >= 2)
10     {
11         k = atoll(argv[1]);
12     }
13
14     if (k <= 0)
15     {
16         fprintf(stderr, "Error: k must be greater than zero.\n");
17         return 1;
18     }
19
20     printf("Value of k: %lld\n\n", k);
21
22     /* Sequential execution */
23     double sequential_start = omp_get_wtime();
24
25     for (long long i = 0; i < k; i++)
26     {
27         printf("Sequential: Hello World, iteration %lld\n", i);
28     }
29
30     double sequential_end = omp_get_wtime();
31     double parallel_start = omp_get_wtime();
32
33     #pragma omp parallel for
34     for (long long i = 0; i < k; i++)
35     {
36         int thread_id = omp_get_thread_num();
37
38         printf(
39             "Parallel: Hello World, iteration %lld, thread %d\n",
40             i,
41             thread_id
42         );
43     }
44
45     double parallel_end = omp_get_wtime();
46
47     double sequential_time = sequential_end - sequential_start;
48     double parallel_time = parallel_end - parallel_start;
49
50     printf("\nSequential execution time: %.9f seconds\n",
51           sequential_time);
52
53     printf("Parallel execution time: %.9f seconds\n",
54           parallel_time);
55
56     if (parallel_time > 0.0)
57     {
58         printf("Speedup: %.4f\n", sequential_time / parallel_time);
59     }
60
61     return 0;
62 }
63
```

Output

```
[vini@asus SEM5]$ gcc -fopenmp number_of_threads.c -o number_of_threads
OMP_NUM_THREADS=8 ./number_of_threads
Number of threads: 8
[vini@asus SEM5]$ OMP_NUM_THREADS=8 ./number_of_threads
Number of threads: 8
[vini@asus SEM5]$ OMP_NUM_THREADS=4 ./number_of_threads
Number of threads: 4
[vini@asus SEM5]$
```

Q3.

A.

```
C: hello_timing.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  int main(int argc, char *argv[])
6  {
7      long long k = 10;
8
9      if (argc >= 2)
10     {
11         k = atoll(argv[1]);
12     }
13
14     if (k <= 0)
15     {
16         fprintf(stderr, "Error: k must be greater than zero.\n");
17         return 1;
18     }
19
20     printf("Value of k: %lld\n\n", k);
21
22     /* Sequential execution */
23     double sequential_start = omp_get_wtime();
24
25     for (long long i = 0; i < k; i++)
26     {
27         printf("Sequential: Hello World, iteration %lld\n", i);
28     }
29
30     double sequential_end = omp_get_wtime();
31     double parallel_start = omp_get_wtime();
32
33     #pragma omp parallel for
34     for (long long i = 0; i < k; i++)
35     {
36         int thread_id = omp_get_thread_num();
37
38         printf(
39             "Parallel: Hello World, iteration %lld, thread %d\n",
40             i,
41             thread_id
42         );
43     }
44
45     double parallel_end = omp_get_wtime();
46
47     double sequential_time = sequential_end - sequential_start;
48     double parallel_time = parallel_end - parallel_start;
49
50     printf("\nSequential execution time: %.9f seconds\n",
51           sequential_time);
52
53     printf("Parallel execution time:   %.9f seconds\n",
54           parallel_time);
55
56     if (parallel_time > 0.0)
57     {
58         printf("Speedup: %.4f\n", sequential_time / parallel_time);
59     }
60
61     return 0;
62 }
```

Output

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
• [vini@asus SEM5]$ gcc -fopenmp hello_timing.c -o hello_timing
OMP_NUM_THREADS=4 ./hello_timing 10
Value of k: 10

Sequential: Hello World, iteration 0
Sequential: Hello World, iteration 1
Sequential: Hello World, iteration 2
Sequential: Hello World, iteration 3
Sequential: Hello World, iteration 4
Sequential: Hello World, iteration 5
Sequential: Hello World, iteration 6
Sequential: Hello World, iteration 7
Sequential: Hello World, iteration 8
Sequential: Hello World, iteration 9
Parallel: Hello World, iteration 8, thread 3
Parallel: Hello World, iteration 9, thread 3
Parallel: Hello World, iteration 6, thread 2
Parallel: Hello World, iteration 7, thread 2
Parallel: Hello World, iteration 0, thread 0
Parallel: Hello World, iteration 1, thread 0
Parallel: Hello World, iteration 2, thread 0
Parallel: Hello World, iteration 3, thread 1
Parallel: Hello World, iteration 4, thread 1
Parallel: Hello World, iteration 5, thread 1

Sequential execution time: 0.000013154 seconds
Parallel execution time: 0.000374750 seconds
Speedup: 0.0351
```

Q4.

```
matrix_multiplication.cpp > ...
1 #include <cmath>
2 #include <cstdlib>
3 #include <iomanip>
4 #include <iostream>
5 #include <random>
6 #include <vector>
7
8 #include <omp.h>
9
10 using Matrix = std::vector<double>;
11
12 void initializeMatrix(Matrix& matrix, int dimension)
13 {
14     std::mt19937 generator(42);
15     std::uniform_real_distribution<double> distribution(0.0, 10.0);
16     for (double& value : matrix)
17     {
18         value = distribution(generator);
19     }
20 }
21
22
23 void serialMatrixMultiplication(
24     const Matrix& matrixA,
25     const Matrix& matrixB,
26     Matrix& result,
27     int dimension
28 )
29 {
30     std::fill(result.begin(), result.end(), 0.0);
31
32     /*
33      * Loop order i-k-j generally provides better cache performance
34      * than the straightforward i-j-k order.
35      */
36     for (int i = 0; i < dimension; ++i)
37     {
38         for (int k = 0; k < dimension; ++k)
39         {
40             const double valueA = matrixA[i * dimension + k];
41             for (int j = 0; j < dimension; ++j)
42             {
43                 result[i * dimension + j] +=
44                     valueA * matrixB[k * dimension + j];
45             }
46         }
47     }
48 }
49
50
51 void parallelMatrixMultiplication(
52     const Matrix& matrixA,
53     const Matrix& matrixB,
54     Matrix& result,
55     int dimension
56 )
57 {
58     std::fill(result.begin(), result.end(), 0.0);
59
60     /*
61      * Each thread processes different rows of the result matrix.
62      * Therefore, threads do not modify the same result elements.
63      */
64     #pragma omp parallel for schedule(static)
65     for (int i = 0; i < dimension; ++i)
66     {
67         for (int k = 0; k < dimension; ++k)
68         {
69             const double valueA = matrixA[i * dimension + k];
```

```

70         for (int j = 0; j < dimension; ++j)
71         {
72             result[i * dimension + j] +=
73                 valueA * matrixB[k * dimension + j];
74         }
75     }
76 }
77 }
78 }
79
80 bool verifyResults(
81     const Matrix& serialResult,
82     const Matrix& parallelResult
83 )
84 {
85     constexpr double tolerance = 1e-8;
86
87     for (std::size_t i = 0; i < serialResult.size(); ++i)
88     {
89         if (std::fabs(serialResult[i] - parallelResult[i]) > tolerance)
90         {
91             std::cerr << "Mismatch at index " << i << '\n';
92             return false;
93         }
94     }
95
96     return true;
97 }
98
99 int main(int argc, char* argv[])
100 {
101     int dimension = 100;
102
103     if (argc >= 2)
104     {
105         dimension = std::atoi(argv[1]);
106     }
107
108     if (dimension <= 0)
109     {
110         std::cerr << "Error: matrix dimension must be positive.\n";
111         return 1;
112     }
113
114     const std::size_t totalElements =
115         static_cast<std::size_t>(dimension) * dimension;
116
117     try
118     {
119         Matrix matrixA(totalElements);
120         Matrix matrixB(totalElements);
121         Matrix serialResult(totalElements);
122         Matrix parallelResult(totalElements);
123
124         initializeMatrix(matrixA, dimension);
125         initializeMatrix(matrixB, dimension);
126
127         std::cout << "Matrix dimension: " << dimension
128             << " x " << dimension << '\n';
129         std::cout << "Number of OpenMP threads: "
130             << omp_get_max_threads() << "\n\n";
131
132         double serialStart = omp_get_wtime();
133
134         serialMatrixMultiplication(
135             matrixA,

```

```

136             matrixB,
137             serialResult,
138             dimension
139         );
140
141         double serialEnd = omp_get_wtime();
142         double parallelStart = omp_get_wtime();
143
144         parallelMatrixMultiplication(
145             matrixA,
146             matrixB,
147             parallelResult,
148             dimension
149         );
150
151         double parallelEnd = omp_get_wtime();
152
153         const double serialTime = serialEnd - serialStart;
154         const double parallelTime = parallelEnd - parallelStart;
155
156         const bool correct =
157             verifyResults(serialResult, parallelResult);
158
159         std::cout << std::fixed << std::setprecision(6);
160
161         std::cout << "Serial execution time: "
162             << serialTime << " seconds\n";
163
164         std::cout << "Parallel execution time: "
165             << parallelTime << " seconds\n";
166
167         if (parallelTime > 0.0)
168         {
169             std::cout << "Speedup: "
170                 << serialTime / parallelTime
171                 << " times\n";
172         }
173
174         std::cout << "Verification: "
175             << (correct ? "PASSED" : "FAILED")
176             << '\n';
177
178         return correct ? 0 : 1;
179     }
180     catch (const std::bad_alloc&)
181     {
182         std::cerr << "Error: insufficient memory for dimension "
183             << dimension << ".\n";
184         return 1;
185     }
186 }

```

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[vini@asus SEM5]$ g++ -O3 -Wall -Wextra -fopenmp matrix_multiplication.cpp -o matrix_multiplication
OMP_NUM_THREADS=4 ./matrix_multiplication 100
Verification: PASSED
• [vini@asus SEM5]$ g++ -O3 -Wall -Wextra -fopenmp matrix_multiplication.cpp -o matrix_multiplication
• [vini@asus SEM5]$ OMP_NUM_THREADS=4 ./matrix_multiplication 100
Matrix dimension: 100 x 100
Number of OpenMP threads: 4

Serial execution time: 0.000468 seconds
Parallel execution time: 0.000425 seconds
Speedup: 1.101414 times
Verification: PASSED
○ [vini@asus SEM5]$
```

Q5.

A.

```
merge_sort.cpp > ...
1 #include <algorithm>
2 #include <cstdlib>
3 #include <iomanip>
4 #include <iostream>
5 #include <random>
6 #include <vector>
7
8 #include <omp.h>
9
10 constexpr int TASK_CUTOFF = 10000;
11
12 void merge(
13     std::vector<int>& values,
14     std::vector<int>& temporary,
15     int left,
16     int middle,
17     int right
18 )
19 {
20     int leftIndex = left;
21     int rightIndex = middle;
22     int outputIndex = left;
23
24     while (leftIndex < middle && rightIndex < right)
25     {
26         if (values[leftIndex] <= values[rightIndex])
27         {
28             temporary[outputIndex++] = values[leftIndex++];
29         }
30         else
31         {
32             temporary[outputIndex++] = values[rightIndex++];
33         }
34     }
35
36     while (leftIndex < middle)
37     {
38         temporary[outputIndex++] = values[leftIndex++];
39     }
40
41     while (rightIndex < right)
42     {
43         temporary[outputIndex++] = values[rightIndex++];
44     }
45
46     for (int i = left; i < right; ++i)
47     {
48         values[i] = temporary[i];
49     }
50 }
51
52 void serialMergeSort(
53     std::vector<int>& values,
54     std::vector<int>& temporary,
55     int left,
56     int right
57 )
58 {
59     if (right - left <= 1)
60     {
61         return;
62     }
63
64     const int middle = left + (right - left) / 2;
65
66     serialMergeSort(values, temporary, left, middle);
67     serialMergeSort(values, temporary, middle, right);
68
69     merge(values, temporary, left, middle, right);
70 }
```

```

57 }
58 }
59
60 void parallelMergeSortTask(
61     std::vector<int>& values,
62     std::vector<int>& temporary,
63     int left,
64     int right
65 )
66 {
67     if (right - left <= 1)
68     {
69         return;
70     }
71
72     if (right - left <= TASK_CUTOFF)
73     {
74         serialMergeSort(values, temporary, left, right);
75         return;
76     }
77
78     const int middle = left + (right - left) / 2;
79
80     #pragma omp task shared(values, temporary) firstprivate(left, middle)
81     {
82         parallelMergeSortTask(
83             values,
84             temporary,
85             left,
86             middle
87         );
88     }
89
90     #pragma omp task shared(values, temporary) firstprivate(middle, right)
91     {
92         parallelMergeSortTask(
93             values,
94             temporary,
95             middle,
96             right
97         );
98     }
99
100     #pragma omp taskwait
101
102     merge(values, temporary, left, middle, right);
103 }
104
105 void parallelMergeSort(std::vector<int>& values)
106 {
107     std::vector<int> temporary(values.size());
108
109     #pragma omp parallel
110     {
111         #pragma omp single
112         {
113             parallelMergeSortTask(
114                 values,
115                 temporary,
116                 0,
117                 static_cast<int>(values.size())
118             );
119         }
120     }
121 }
122
123 std::vector<int> generateRandomValues(int count)
124 {
125     std::vector<int> values(count);
126
127     for (int i = 0; i < count; i++)
128     {
129         values[i] = rand() % 1000000;
130     }
131 }

```

```

135 std::vector<int> generateRandomValues(int count)
136 {
137     std::mt19937 generator(42);
138     std::uniform_int_distribution<int> distribution(
139         0,
140         1'000'000
141     );
142     for (int& value : values)
143     {
144         value = distribution(generator);
145     }
146     return values;
147 }
148
149 int main(int argc, char* argv[])
150 {
151     int numberOfElements = 1'000'000;
152
153     if (argc >= 2)
154     {
155         numberOfElements = std::atoi(argv[1]);
156     }
157
158     if (numberOfElements <= 0)
159     {
160         std::cerr << "Error: N must be positive.\n";
161         return 1;
162     }
163
164     std::vector<int> original =
165         generateRandomValues(numberOfElements);
166
167     std::vector<int> serialValues = original;
168     std::vector<int> parallelValues = original;
169
170     std::vector<int> serialTemporary(numberOfElements);
171
172     std::cout << "Number of elements: "
173         << numberOfElements << '\n';
174
175     std::cout << "OpenMP threads: "
176         << omp_get_max_threads() << "\n\n";
177
178     double serialStart = omp_get_wtime();
179
180     serialMergeSort(
181         serialValues,
182         serialTemporary,
183         0,
184         numberOfElements
185     );
186
187     double serialEnd = omp_get_wtime();
188
189     double parallelStart = omp_get_wtime();
190
191     parallelMergeSort(parallelValues);
192
193     double parallelEnd = omp_get_wtime();
194
195     const double serialTime = serialEnd - serialStart;
196     const double parallelTime = parallelEnd - parallelStart;
197
198     const bool serialSorted =
199         std::is_sorted(serialValues.begin(), serialValues.end());
200
201     const bool parallelSorted =
202         std::is_sorted(parallelValues.begin(), parallelValues.end());

```

```

203
204     const bool identical =
205         serialValues == parallelValues;
206
207     std::cout << std::fixed << std::setprecision(6);
208
209     std::cout << "Serial merge sort time: "
210         << serialTime << " seconds\n";
211
212     std::cout << "Parallel merge sort time: "
213         << parallelTime << " seconds\n";
214
215     if (parallelTime > 0.0)
216     {
217         std::cout << "Speedup: "
218             << serialTime / parallelTime
219             << " times\n";
220     }
221
222     std::cout << "Serial sorted: "
223         << (serialSorted ? "YES" : "NO") << '\n';
224
225     std::cout << "Parallel sorted: "
226         << (parallelSorted ? "YES" : "NO") << '\n';
227
228     std::cout << "Results identical: "
229         << (identical ? "YES" : "NO") << '\n';
230
231     return serialSorted && parallelSorted && identical ? 0 : 1;
232 }
233
234
235

```


Output

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

[vini@asus SEM5]$ g++ -O3 -Wall -Wextra -fopenmp merge_sort.cpp -o merge_sort
OMP_NUM_THREADS=4 ./merge_sort 100000
Number of elements: 100000
OpenMP threads: 4

Serial merge sort time: 0.017314 seconds
Parallel merge sort time: 0.007368 seconds
Speedup: 2.349840 times
Serial sorted: YES
Parallel sorted: YES
Results identical: YES
[vini@asus SEM5]$
```

5b.

```
quick_sort.cpp > ...
1 #include <algorithm>
2 #include <cstdlib>
3 #include <iomanip>
4 #include <iostream>
5 #include <random>
6 #include <vector>
7
8 #include <omp.h>
9
10 constexpr int TASK_CUTOFF = 10000;
11 constexpr int INSERTION_SORT_CUTOFF = 32;
12
13 void insertionSort(
14     std::vector<int>& values,
15     int low,
16     int high
17 )
18 {
19     for (int i = low + 1; i <= high; ++i)
20     {
21         const int current = values[i];
22         int j = i - 1;
23
24         while (j >= low && values[j] > current)
25         {
26             values[j + 1] = values[j];
27             --j;
28         }
29
30         values[j + 1] = current;
31     }
32 }
33
34 int partitionValues(
35     std::vector<int>& values,
36     int low,
37     int high
38 )
39 {
40     const int middle = low + (high - low) / 2;
41
42     /*
43      * Median-of-three ordering improves pivot selection compared
44      * with always choosing the first or last element.
45      */
46     if (values[middle] < values[low])
47     {
48         std::swap(values[middle], values[low]);
49     }
50
51     if (values[high] < values[low])
52     {
53         std::swap(values[high], values[low]);
54     }
55
56     if (values[high] < values[middle])
57     {
58         std::swap(values[high], values[middle]);
59     }
60
61     std::swap(values[middle], values[high - 1]);
62
63     const int pivot = values[high - 1];
64
65     int left = low;
66     int right = high - 1;
67
68     while (true)
69     {
70         while (values[left] < pivot) ++left;
71         while (values[right] > pivot) --right;
72         if (left < right)
73             std::swap(values[left], values[right]);
74     }
75
76     std::swap(values[high - 1], values[left]);
77
78     return left;
79 }
```

```

38     )
68     while (true)
70     {
71         while (values[++left] < pivot)
72         {
73             ;
74         }
75         while (values[--right] > pivot)
76         {
77             ;
78         }
79         if (left >= right)
80         {
81             break;
82         }
83         std::swap(values[left], values[right]);
84     }
85     std::swap(values[left], values[high - 1]);
86     return left;
87 }
88
89
90
91 void serialQuickSort(
92     std::vector<int>& values,
93     int low,
94     int high
95 )
96 {
97     if (low >= high)
98     {
99         return;
100     }
101
102     if (high - low + 1 <= INSERTION_SORT_CUTOFF)
103     {
104         insertionSort(values, low, high);
105         return;
106     }
107
108     const int pivotIndex =
109         partitionValues(values, low, high);
110
111     serialQuickSort(values, low, pivotIndex - 1);
112     serialQuickSort(values, pivotIndex + 1, high);
113 }
114
115 void parallelQuickSortTask(
116     std::vector<int>& values,
117     int low,
118     int high
119 )
120 {
121     if (low >= high)
122     {
123         return;
124     }
125
126     const int partitionSize = high - low + 1;
127
128     if (partitionSize <= TASK_CUTOFF)
129     {
130         serialQuickSort(values, low, high);
131         return;
132     }
133
134     const int pivotIndex =
135         partitionValues(values, low, high);
136
137     ;
138     ;
139     ;
140     ;
141     ;
142     ;
143     ;
144     ;
145     ;
146     ;
147     ;
148     ;
149     ;
150     ;
151     ;
152     ;
153     ;
154     ;
155     ;
156     ;
157     ;
158     ;
159     ;
160     ;
161     ;
162     ;
163     ;
164     ;
165     ;
166     ;
167     ;
168     ;
169     ;
170     ;
171     ;
172     ;
173     ;
174     ;
175     ;
176     ;
177     ;
178     ;
179     ;
180     ;
181     ;
182     ;
183     ;
184     ;
185     ;
186     ;
187     ;
188     ;
189     ;
190     ;
191     ;
192     ;
193     ;
194     ;
195     ;
196     ;
197     ;
198     ;
199     ;
200     ;
201     ;
202     ;
203     ;
204     ;
205     ;
206     ;
207     ;
208     ;
209     ;
210     ;
211     ;
212     ;
213     ;
214     ;
215     ;
216     ;
217     ;
218     ;
219     ;
220     ;
221     ;
222     ;
223     ;
224     ;
225     ;
226     ;
227     ;
228     ;
229     ;
230     ;
231     ;
232     ;
233     ;
234     ;
235     ;
236     ;
237     ;
238     ;
239     ;
240     ;
241     ;
242     ;
243     ;
244     ;
245     ;
246     ;
247     ;
248     ;
249     ;
250     ;
251     ;
252     ;
253     ;
254     ;
255     ;
256     ;
257     ;
258     ;
259     ;
260     ;
261     ;
262     ;
263     ;
264     ;
265     ;
266     ;
267     ;
268     ;
269     ;
270     ;
271     ;
272     ;
273     ;
274     ;
275     ;
276     ;
277     ;
278     ;
279     ;
280     ;
281     ;
282     ;
283     ;
284     ;
285     ;
286     ;
287     ;
288     ;
289     ;
290     ;
291     ;
292     ;
293     ;
294     ;
295     ;
296     ;
297     ;
298     ;
299     ;
300     ;
301     ;
302     ;
303     ;
304     ;
305     ;
306     ;
307     ;
308     ;
309     ;
310     ;
311     ;
312     ;
313     ;
314     ;
315     ;
316     ;
317     ;
318     ;
319     ;
320     ;
321     ;
322     ;
323     ;
324     ;
325     ;
326     ;
327     ;
328     ;
329     ;
330     ;
331     ;
332     ;
333     ;
334     ;
335     ;
336     ;
337     ;
338     ;
339     ;
340     ;
341     ;
342     ;
343     ;
344     ;
345     ;
346     ;
347     ;
348     ;
349     ;
350     ;
351     ;
352     ;
353     ;
354     ;
355     ;
356     ;
357     ;
358     ;
359     ;
360     ;
361     ;
362     ;
363     ;
364     ;
365     ;
366     ;
367     ;
368     ;
369     ;
370     ;
371     ;
372     ;
373     ;
374     ;
375     ;
376     ;
377     ;
378     ;
379     ;
380     ;
381     ;
382     ;
383     ;
384     ;
385     ;
386     ;
387     ;
388     ;
389     ;
390     ;
391     ;
392     ;
393     ;
394     ;
395     ;
396     ;
397     ;
398     ;
399     ;
400     ;
401     ;
402     ;
403     ;
404     ;
405     ;
406     ;
407     ;
408     ;
409     ;
410     ;
411     ;
412     ;
413     ;
414     ;
415     ;
416     ;
417     ;
418     ;
419     ;
420     ;
421     ;
422     ;
423     ;
424     ;
425     ;
426     ;
427     ;
428     ;
429     ;
430     ;
431     ;
432     ;
433     ;
434     ;
435     ;
436     ;
437     ;
438     ;
439     ;
440     ;
441     ;
442     ;
443     ;
444     ;
445     ;
446     ;
447     ;
448     ;
449     ;
450     ;
451     ;
452     ;
453     ;
454     ;
455     ;
456     ;
457     ;
458     ;
459     ;
460     ;
461     ;
462     ;
463     ;
464     ;
465     ;
466     ;
467     ;
468     ;
469     ;
470     ;
471     ;
472     ;
473     ;
474     ;
475     ;
476     ;
477     ;
478     ;
479     ;
480     ;
481     ;
482     ;
483     ;
484     ;
485     ;
486     ;
487     ;
488     ;
489     ;
490     ;
491     ;
492     ;
493     ;
494     ;
495     ;
496     ;
497     ;
498     ;
499     ;
500     ;
501     ;
502     ;
503     ;
504     ;
505     ;
506     ;
507     ;
508     ;
509     ;
510     ;
511     ;
512     ;
513     ;
514     ;
515     ;
516     ;
517     ;
518     ;
519     ;
520     ;
521     ;
522     ;
523     ;
524     ;
525     ;
526     ;
527     ;
528     ;
529     ;
530     ;
531     ;
532     ;
533     ;
534     ;
535     ;
536     ;
537     ;
538     ;
539     ;
540     ;
541     ;
542     ;
543     ;
544     ;
545     ;
546     ;
547     ;
548     ;
549     ;
550     ;
551     ;
552     ;
553     ;
554     ;
555     ;
556     ;
557     ;
558     ;
559     ;
560     ;
561     ;
562     ;
563     ;
564     ;
565     ;
566     ;
567     ;
568     ;
569     ;
570     ;
571     ;
572     ;
573     ;
574     ;
575     ;
576     ;
577     ;
578     ;
579     ;
580     ;
581     ;
582     ;
583     ;
584     ;
585     ;
586     ;
587     ;
588     ;
589     ;
590     ;
591     ;
592     ;
593     ;
594     ;
595     ;
596     ;
597     ;
598     ;
599     ;
600     ;
601     ;
602     ;
603     ;
604     ;
605     ;
606     ;
607     ;
608     ;
609     ;
610     ;
611     ;
612     ;
613     ;
614     ;
615     ;
616     ;
617     ;
618     ;
619     ;
620     ;
621     ;
622     ;
623     ;
624     ;
625     ;
626     ;
627     ;
628     ;
629     ;
630     ;
631     ;
632     ;
633     ;
634     ;
635     ;
636     ;
637     ;
638     ;
639     ;
640     ;
641     ;
642     ;
643     ;
644     ;
645     ;
646     ;
647     ;
648     ;
649     ;
650     ;
651     ;
652     ;
653     ;
654     ;
655     ;
656     ;
657     ;
658     ;
659     ;
660     ;
661     ;
662     ;
663     ;
664     ;
665     ;
666     ;
667     ;
668     ;
669     ;
670     ;
671     ;
672     ;
673     ;
674     ;
675     ;
676     ;
677     ;
678     ;
679     ;
680     ;
681     ;
682     ;
683     ;
684     ;
685     ;
686     ;
687     ;
688     ;
689     ;
690     ;
691     ;
692     ;
693     ;
694     ;
695     ;
696     ;
697     ;
698     ;
699     ;
700     ;
701     ;
702     ;
703     ;
704     ;
705     ;
706     ;
707     ;
708     ;
709     ;
710     ;
711     ;
712     ;
713     ;
714     ;
715     ;
716     ;
717     ;
718     ;
719     ;
720     ;
721     ;
722     ;
723     ;
724     ;
725     ;
726     ;
727     ;
728     ;
729     ;
730     ;
731     ;
732     ;
733     ;
734     ;
735     ;
736     ;
737     ;
738     ;
739     ;
740     ;
741     ;
742     ;
743     ;
744     ;
745     ;
746     ;
747     ;
748     ;
749     ;
750     ;
751     ;
752     ;
753     ;
754     ;
755     ;
756     ;
757     ;
758     ;
759     ;
760     ;
761     ;
762     ;
763     ;
764     ;
765     ;
766     ;
767     ;
768     ;
769     ;
770     ;
771     ;
772     ;
773     ;
774     ;
775     ;
776     ;
777     ;
778     ;
779     ;
780     ;
781     ;
782     ;
783     ;
784     ;
785     ;
786     ;
787     ;
788     ;
789     ;
790     ;
791     ;
792     ;
793     ;
794     ;
795     ;
796     ;
797     ;
798     ;
799     ;
800     ;
801     ;
802     ;
803     ;
804     ;
805     ;
806     ;
807     ;
808     ;
809     ;
810     ;
811     ;
812     ;
813     ;
814     ;
815     ;
816     ;
817     ;
818     ;
819     ;
820     ;
821     ;
822     ;
823     ;
824     ;
825     ;
826     ;
827     ;
828     ;
829     ;
830     ;
831     ;
832     ;
833     ;
834     ;
835     ;
836     ;
837     ;
838     ;
839     ;
840     ;
841     ;
842     ;
843     ;
844     ;
845     ;
846     ;
847     ;
848     ;
849     ;
850     ;
851     ;
852     ;
853     ;
854     ;
855     ;
856     ;
857     ;
858     ;
859     ;
860     ;
861     ;
862     ;
863     ;
864     ;
865     ;
866     ;
867     ;
868     ;
869     ;
870     ;
871     ;
872     ;
873     ;
874     ;
875     ;
876     ;
877     ;
878     ;
879     ;
880     ;
881     ;
882     ;
883     ;
884     ;
885     ;
886     ;
887     ;
888     ;
889     ;
890     ;
891     ;
892     ;
893     ;
894     ;
895     ;
896     ;
897     ;
898     ;
899     ;
900     ;
901     ;
902     ;
903     ;
904     ;
905     ;
906     ;
907     ;
908     ;
909     ;
910     ;
911     ;
912     ;
913     ;
914     ;
915     ;
916     ;
917     ;
918     ;
919     ;
920     ;
921     ;
922     ;
923     ;
924     ;
925     ;
926     ;
927     ;
928     ;
929     ;
930     ;
931     ;
932     ;
933     ;
934     ;
935     ;
936     ;
937     ;
938     ;
939     ;
940     ;
941     ;
942     ;
943     ;
944     ;
945     ;
946     ;
947     ;
948     ;
949     ;
950     ;
951     ;
952     ;
953     ;
954     ;
955     ;
956     ;
957     ;
958     ;
959     ;
960     ;
961     ;
962     ;
963     ;
964     ;
965     ;
966     ;
967     ;
968     ;
969     ;
970     ;
971     ;
972     ;
973     ;
974     ;
975     ;
976     ;
977     ;
978     ;
979     ;
980     ;
981     ;
982     ;
983     ;
984     ;
985     ;
986     ;
987     ;
988     ;
989     ;
990     ;
991     ;
992     ;
993     ;
994     ;
995     ;
996     ;
997     ;
998     ;
999     ;
1000    ;

```

```

119 )
136
137 #pragma omp task shared(values) firstprivate(low, pivotIndex)
138 {
139     parallelQuickSortTask(
140         values,
141         low,
142         pivotIndex - 1
143     );
144 }
145
146 #pragma omp task shared(values) firstprivate(high, pivotIndex)
147 {
148     parallelQuickSortTask(
149         values,
150         pivotIndex + 1,
151         high
152     );
153 }
154
155 #pragma omp taskwait
156 }
157
158 void parallelQuickSort(std::vector<int>& values)
159 {
160     #pragma omp parallel
161     {
162         #pragma omp single
163         {
164             parallelQuickSortTask(
165                 values,
166                 0,
167                 static_cast<int>(values.size()) - 1
168             );
169         }
170     }
171 }
172
173 std::vector<int> generateRandomValues(int count)
174 {
175     std::vector<int> values(count);
176
177     std::mt19937 generator(42);
178     std::uniform_int_distribution<int> distribution(
179         0,
180         1'000'000
181     );
182
183     for (int& value : values)
184     {
185         value = distribution(generator);
186     }
187
188     return values;
189 }
190
191 int main(int argc, char* argv[])
192 {
193     int numberOfElements = 1'000'000;
194
195     if (argc >= 2)
196     {
197         numberOfElements = std::atoi(argv[1]);
198     }
199
200     if (numberOfElements <= 0)
201     {
202         std::cerr << "Error: N must be positive.\n";
203         return 1;
204     }
}

```

```

191 int main(int argc, char* argv[])
205
206     std::vector<int> original =
207         generateRandomValues(numberOfElements);
208
209     std::vector<int> serialValues = original;
210     std::vector<int> parallelValues = original;
211
212     std::cout << "Number of elements: "
213         << numberOfElements << '\n';
214
215     std::cout << "OpenMP threads: "
216         << omp_get_max_threads() << "\n\n";
217
218     double serialStart = omp_get_wtime();
219
220     serialQuickSort(
221         serialValues,
222         0,
223         numberOfElements - 1
224     );
225
226     double serialEnd = omp_get_wtime();
227
228     double parallelStart = omp_get_wtime();
229
230     parallelQuickSort(parallelValues);
231
232     double parallelEnd = omp_get_wtime();
233
234     const double serialTime = serialEnd - serialStart;
235     const double parallelTime = parallelEnd - parallelStart;
236
237     const bool serialSorted =
238         std::is_sorted(serialValues.begin(), serialValues.end());
239
240     const bool parallelSorted =
241         std::is_sorted(parallelValues.begin(), parallelValues.end());
242
243     const bool identical =
244         serialValues == parallelValues;
245
246     std::cout << std::fixed << std::setprecision(6);
247
248     std::cout << "Serial quick sort time: "
249         << serialTime << " seconds\n";
250
251     std::cout << "Parallel quick sort time: "
252         << parallelTime << " seconds\n";
253
254     if (parallelTime > 0.0)
255     {
256         std::cout << "Speedup: "
257             << serialTime / parallelTime
258             << " times\n";
259     }
260
261     std::cout << "Serial sorted: "
262         << (serialSorted ? "YES" : "NO") << '\n';
263
264     std::cout << "Parallel sorted: "
265         << (parallelSorted ? "YES" : "NO") << '\n';
266
267     std::cout << "Results identical: "
268         << (identical ? "YES" : "NO") << '\n';
269
270     return serialSorted && parallelSorted && identical ? 0 : 1;
271 }
272

```

Output

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

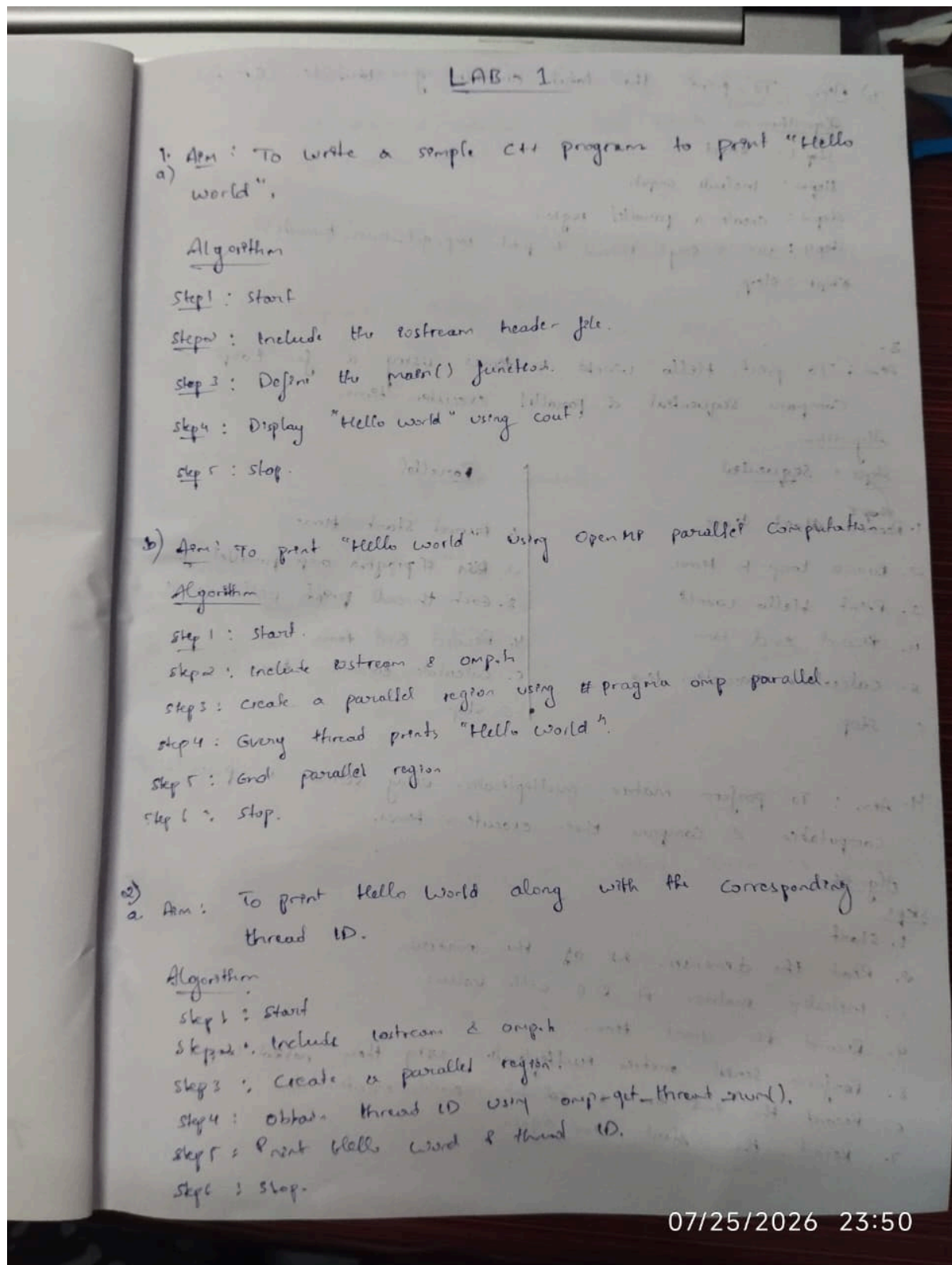
[vini@asus SEM5]$ OMP_NUM_THREADS=4 ./matrix_multiplication 100
Parallel execution time: 0.000425 seconds
Speedup: 1.101414 times
Verification: PASSED
• [vini@asus SEM5]$ g++ -O3 -Wall -Wextra -fopenmp merge_sort.cpp -o merge_sort
OMP_NUM_THREADS=4 ./merge_sort 100000
Number of elements: 100000
OpenMP threads: 4

Serial merge sort time: 0.017314 seconds
Parallel merge sort time: 0.007368 seconds
Speedup: 2.349840 times
Serial sorted: YES
Parallel sorted: YES
Results identical: YES
• [vini@asus SEM5]$ g++ -O3 -Wall -Wextra -fopenmp quick_sort.cpp -o quick_sort
OMP_NUM_THREADS=4 ./quick_sort 100000
Number of elements: 100000
OpenMP threads: 4

Serial quick sort time: 0.011869 seconds
Parallel quick sort time: 0.003655 seconds
Speedup: 3.247554 times
Serial sorted: YES
Parallel sorted: YES
Results identical: YES
○ [vini@asus SEM5]$

```

Screenshots of algorithms



07/25/2026 23:50

b) Aim: To print the total number of threads created

Algorithm

- Step 1: Start of program
- Step 2: Include omp.h
- Step 3: Create a parallel region.
- Step 4: Use a single thread to print omp_get_num_threads().
- Step 5: Stop.

3. Aim: To print Hello world K times using a for loop & compare sequential & parallel execution time.

Algorithm

Steps: Sequential

1. Record start time.
2. Run a loop to times.
3. Print Hello world
4. Record end time.
5. Calculate execution time.
6. Stop.

Parallel

1. Record start time.
2. Run #pragma omp parallel for.
3. Each thread prints Hello world.
4. Record end time.
5. Calculate execution time.
6. Stop.

4. Aim: To perform matrix multiplication using serial & parallel computation & compare their execution times.

Algorithm

Steps

1. Start.
2. Read the dimension N of the matrices.
3. Initialize matrix A & B with values.
4. Record the start time.
5. Perform serial matrix multiplication using three nested loops.
6. Record the end time & calculate serial execution time.
7. Record the start time again.

07/25/2026 23:50

8. Perform parallel, sequential, multiprocessor using OpenMP.
9. Record the end time & calculate parallel execution time.
10. Display both execution times.
11. Stop.

5>

a) Aim: To implement Merge Sort using serial & parallel computation & compare their execution times.

Algorithm

Step

1. Start
2. Generate an array of N elements.
3. Record the start time.
4. Perform serial Merge Sort.
5. Record the end time.
6. Record the start time again.
7. Perform parallel Merge Sort using OpenMP tasks/sections.
8. Record the end time.
9. Display both execution times.
10. Stop.

b) Aim: To implement Quick Sort using serial & parallel computation & compare their execution times.

Algorithm

1. Start
2. Generate an array of N elements
3. Record the start time.
4. Perform ~~the end time~~ serial Quick Sort.
5. Record the end time
6. Record the start time again.

07/25/2026 23:50

7. Perform parallel quick sort using OpenMP tasks.
8. Record the end time.
9. Display both execution times.
10. Stop.

07/25/2026 23:50